# AICTE - EDUSKILLS VIRTUAL INTERNSHIP

# PROJECT REPORT

## "Text Encryption Toolkit"

**SUBMITTED BY:**

Student Name: P N Kavitha
Roll Number: N/A
Program: B.Tech
Branch: Computer Science

**SUBMITTED TO:**

**EduSkills Academy**

**Internship Details:**

Internship Duration: 0 Weeks (Jun 2026 - Jun 2026)
Supported by: Eduskills Academy

**Under the Guidance of**

Project Manager
Eduskills

# TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Mr./Ms. P N Kavitha**, Roll No. **N/A**, a student of **EduSkills Academy**, has successfully completed **0-Week (Jun 2026 - Jun 2026)** AICTE – EduSkills Virtual Internship Program.

During the internship, he/she worked on the project titled **"Text Encryption Toolkit"**. His/her performance during the internship was found to be satisfactory, and he/she demonstrated good technical and professional skills.

We wish him/her success in future endeavors.

**Sagar Bala Behera**
Mentor, EduSkills Academy
Date: 10-06-2026

---

## DECLARATION

I hereby declare that the work presented in this Online Virtual Internship Project Report is my original work carried out under the guidance of my industry mentor and faculty guide. This work has not been submitted earlier for the award of any degree or diploma.

**P N Kavitha**
(Signature)
Date: 10-06-2026

## ACKNOWLEDGEMENT

I express my sincere gratitude to **AICTE & EduSkills** for providing me with the opportunity to undertake this online virtual internship. I would like to thank my industry mentor **Mr./Ms. Sagar Bala Behera** for his/her valuable guidance, continuous support, and technical insights throughout the internship period.

I am also thankful to the Department of **Computer Science** for their encouragement and support.


---

## ABSTRACT


This project details the development of a comprehensive Text Encryption Toolkit, focusing on implementing classic cryptographic algorithms and utility functions. The toolkit includes features for Caesar cipher encryption and decryption, ROT13, string reversal, letter frequency analysis, and a history manager to track operations. The primary goal is to provide a practical understanding of string manipulation, modular arithmetic, and data structures in Python.

# Table of Contents

## 1. Introduction

Industry-oriented learning through internships plays a crucial role in bridging the gap between academic knowledge and real-world applications. This online virtual internship provided hands-on exposure to **Python Full Stack Development With Project** and its application in modern software solutions.

## 2. Organization Profile

**Organization Name: EduSkills Academy**

**Domain: EdTech & Skill Development**

**Services:** Virtual Internships

**Vision:** To empower students with industry-relevant skills.

## 3. Internship Overview

• Internship Mode: **Online / Virtual**

• Duration: **10 Weeks**

• Platform Used: **EduSkills LMS**

• Role: **Intern**

## 4. Problem Statement & Objectives

**Problem Statement:**

The Text Encryption Toolkit is a software utility designed to provide a versatile set of tools for text manipulation and basic encryption. It offers implementations of historical ciphers such as the Caesar cipher and ROT13, alongside functional utilities for reversing text and analyzing letter frequencies. A key feature is the integrated history manager, which records each operation performed, providing a log for review. This project serves as a practical exploration of fundamental programming concepts, including string processing, algorithmic design, and data management within the Python environment.

**Objectives:**

- To design and implement a solution for the given problem statement.
- To utilize modern tools and algorithms.
- To optimize performance and code quality.

## 5. Project Description & Methodology

The development of the Text Encryption Toolkit was approached systematically. Initially, the core encryption algorithms, including the Caesar cipher (both encryption and decryption), were implemented, leveraging modular arithmetic for character shifting. Subsequently, the ROT13 cipher and string reversal functions were added, building upon basic string manipulation techniques. A letter frequency counter was developed using dictionaries to efficiently tally character occurrences. Finally, an Encryption History Manager was designed to store and track a log of all performed operations, ensuring a record of the toolkit's usage. Each component was developed and tested independently before integration.

## 6. Tools & Technologies Used

Programming Language: Python
Concepts: String Manipulation, Modular Arithmetic, Data Structures (Dictionaries, Lists), Algorithmic Implementation

## 7. Implementation & Results

### Problem 1: Caesar Cipher Encryption

**Description:** This section addresses the implementation of the Caesar cipher for encrypting text. The task involves shifting each letter in the plaintext by a specified number of positions down the alphabet. The solution must correctly handle wrapping around from 'z' to 'a' using modular arithmetic to ensure all characters remain within the alphabet.

```
import sys
import json

input_data = sys.stdin.read()
if input_data.endswith('\n'):
    input_data = input_data[:-1]

lines = input_data.split('\n')

results = []
for line in lines:
    if not line.strip():
        continue
    # TODO: Implement ENCRYPT <shift> <text>
    # result = {"result": "..."}
    parts = line.split()
    if len(parts)< 3:
        continue
    command = parts[0]
    shift = int(parts[1])
    text = parts[2]

encrypted = ""

for ch in text:
    new_pos = (ord(ch) - ord('a') + shift) % 26
    encrypted += chr(new_pos + ord('a'))

    result = {"result": encrypted}
results.append(result)

print(json.dumps(results, separators=(',', ':')))
```

## Problem 2: Caesar Cipher Decryption

**Description:** This section covers the decryption of text encrypted using the Caesar cipher. It requires reversing the encryption process by shifting each letter in the ciphertext back by the same specified number of positions. The implementation should utilize modular arithmetic, effectively performing encryption with a negative shift to restore the original plaintext.

```
import sys
import json

input_data = sys.stdin.read()
if input_data.endswith('\n'):
    input_data = input_data[:-1]

lines = input_data.split('\n')
results = []
for line in lines:
    if not line.strip():
        continue
    # TODO: Implement DECRYPT <shift> <text>
    parts = line.split()

    if len(parts) < 3:
        continue

command = parts[0]
shift = int(parts[1])
text = parts[2]

decrypted = ""

for ch in text:
    new_pos = (ord(ch) - ord('a') - shift) % 26
    decrypted += chr(new_pos + ord('a'))

result = {"result": decrypted}
results.append(result)

print(json.dumps(results, separators=(',', ':')))
```

## Problem 3: ROT13 and Reverse Ciphers

**Description:** This task involves implementing two distinct text transformation methods: ROT13 and string reversal. ROT13 is a specific case of the Caesar cipher where the shift is fixed at 13 positions. String reversal requires reordering the characters of a given text from end to beginning. Both functionalities should be implemented as separate, reusable functions.

```
import sys
import json

input_data = sys.stdin.read()
if input_data.endswith('\n'):
    input_data = input_data[:-1]

lines = input_data.split('\n')
results = []
for line in lines:
    if not line.strip():
        continue
    # TODO: Implement ROT13 and REVERSE commands
    parts = line.split()

command = parts[0]
text = parts[1]

if command == "ROT13":
    result_text = ""
    for ch in text:
        new_pos = (ord(ch) - ord('a') + 13) % 26
        result_text += chr(new_pos + ord('a'))

elif command == "REVERSE":
    result_text = text[::-1]

result = {"result": result_text}
results.append(result)


print(json.dumps(results, separators=(',', ':')))
```

## Problem 4: Letter Frequency Counter

**Description:** This section focuses on developing a tool to count the occurrences of each letter within a given text. The solution should utilize a data structure, such as a dictionary, to store the frequency of each character. The counter needs to iterate through the input text and update the counts for every encountered letter, providing a statistical breakdown of the text's composition.

```
import sys
import json

input_data = sys.stdin.read()
if input_data.endswith('\n'):
    input_data = input_data[:-1]

lines = input_data.split('\n')
results = []
for line in lines:
    if not line.strip():
        continue
    # TODO: Implement COUNT <text>
    parts = line.split()

text = parts[1].lower()

counts = {}

for ch in text:
    if ch.isalpha():
        counts[ch] = counts.get(ch, 0) + 1

result = {"counts": counts}
results.append(result)


print(json.dumps(results, separators=(',', ':')))
```

## Problem 5: Encryption History Manager

**Description:** This task requires the creation of a system to log and manage the history of operations performed by the toolkit. The manager should store details of each operation, including the type of operation performed and its output. A list or similar data structure will be employed to maintain this chronological record of actions.

```
import sys
import json

input_data = sys.stdin.read()
if input_data.endswith('\n'):
    input_data = input_data[:-1]

lines = input_data.split('\n')
history = []

for line in lines:
    if not line.strip():
        continue
    # TODO: Process commands and append to history
    parts = line.split()
    command = parts[0]
    if command == "ENCRYPT":
        shift = int(parts[1])
        text = parts[2]
        result_text = ""
        for ch in text:
            new_pos = (ord(ch) - ord('a') + shift) % 26
            result_text += chr(new_pos + ord('a'))
        history.append({"op":"ENCRYPT","out":result_text})
    elif command == "REVERSE":
        text = parts[1]
        history.append({"op":"REVERSE","out":text[::-1]})

print(json.dumps(history, separators=(',', ':')))
```

## 8. Learning Outcomes

• Gained practical experience in implementing classical encryption algorithms like the Caesar cipher and ROT13.
• Enhanced understanding of string manipulation techniques and their application in cryptography.
• Developed proficiency in using dictionaries for frequency analysis and lists for managing operation history.
• Solidified knowledge of modular arithmetic for handling character shifts and alphabet wrap-around.

## 9. Challenges Faced & Solutions

Challenge: Handling both uppercase and lowercase letters consistently during Caesar cipher encryption/decryption.
Solution: Implemented logic to detect the case of each character and apply the shift within the respective case's alphabet range, ensuring case preservation.

Challenge: Efficiently counting letter frequencies without missing any characters or double-counting.
Solution: Utilized a dictionary to store counts, iterating through the input string and incrementing the count for each character, with a default value of zero for characters encountered for the first time.

Challenge: Ensuring the history manager accurately records operation details in a useful format.
Solution: Designed a structured entry format (e.g., a dictionary) for each operation, including type and output, and appended these entries to a list, allowing for straightforward retrieval and display.

## 10. Conclusion

The Text Encryption Toolkit successfully delivers a set of fundamental text manipulation and encryption tools. By implementing classical ciphers and utility functions, the project provides a practical foundation in cryptography, string processing, and data management. The inclusion of a history manager enhances usability by tracking operations. This toolkit serves as a valuable resource for learning and experimenting with basic cryptographic concepts.

## 11. Future Scope

• Implement more advanced encryption algorithms, such as Vigenère cipher or substitution ciphers.
• Add functionality for analyzing decryption attempts using frequency analysis.
• Develop a graphical user interface (GUI) for improved user interaction and accessibility.

## 12. References

• Python Official Documentation: https://docs.python.org/3/
• Introduction to Cryptography (Online Resources)
• Data Structures and Algorithms in Python (Textbooks/Online Courses)

## 13. Annexure

• Internship Completion Certificate
• Offer Letter